OFFICIAL DOCUMENTATION

VIBEVps

Installation and Usage Manual

VERSION	1.3.1
LAST UPDATED	March 22, 2026
PLATFORM	Proxmox VE Management Dashboard
LICENSE	Open Source

Hypervisor Management • VM Provisioning • Cloud-Init • Web SSH Console

VIBEVps © 2026 — All rights reserved

Table of Contents

1. Introduction
2. System Requirements
3. Installation
    - 3.1 Server Preparation
    - 3.2 Automatic Installation
    - 3.3 Manual Installation
    - 3.4 Environment Configuration (.env)
    - 3.5 Nginx Configuration
    - 3.6 Starting with PM2
4. First Login
5. Proxmox Hypervisor Configuration
    - 5.1 Creating an API Token on Proxmox
    - 5.2 Adding a Hypervisor in VIBEVps
    - 5.3 Connection Test
6. Dashboard
    - 6.1 Node Overview
    - 6.2 VM Statistics
    - 6.3 Storage and Network Traffic
    - 6.4 Hypervisor Filter
7. Virtual Machine Management
    - 7.1 VM List
    - 7.2 VM Details
    - 7.3 VM Actions
    - 7.4 VM Deletion
8. Creating Virtual Machines
    - 8.1 Prerequisites: Proxmox Templates
    - 8.2 Creation Wizard (5 Steps)
    - 8.3 Cloud-Init Configuration
    - 8.4 What Happens After Creation
9. Template Management
    - 9.1 Automatic Template Discovery
    - 9.2 Saving Templates with Default Values
10. Web SSH Console
    - 10.1 Requirements
    - 10.2 Connecting
    - 10.3 IP Resolution
11. Activity Logs
12. Settings

    - 12.1 Password Change
    - 12.2 Storage Visibility
13. Update System
    - 13.1 Checking for Updates
    - 13.2 Installing Updates
    - 13.3 Sequential Updates
    - 13.4 Automatic Backup
14. Technical Architecture
    - 14.1 Technology Stack
    - 14.2 Project Structure
    - 14.3 SQLite Database
    - 14.4 REST API
    - 14.5 Real-time WebSocket
15. Maintenance and Troubleshooting
    - 15.1 Useful PM2 Commands
    - 15.2 System Logs
    - 15.3 Database Backup
    - 15.4 Common Issues and Solutions
16. Security
17. Local Development

---

1. Introduction

VIBEVps is an open-source web dashboard for managing Proxmox VE hypervisors and their virtual machines. It allows you to:

- Monitor Proxmox node status in real time (CPU, RAM, disk, network)
- View and control all VMs (start, stop, restart, suspend, delete)
- Create new VMs from templates with automatic cloud-init configuration (hostname, static IP/DHCP, DNS)
- Access VMs via SSH terminal directly from the browser
- Manage templates, storage, and logs for all operations
- Receive automatic updates without requiring a license

The interface is fully in English, responsive (desktop, tablet, mobile), with a dark theme and real-time updates via WebSocket.

---

2. System Requirements

VIBEVps Server

Requirement	Minimum	Recommended
Operating System	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
CPU	1 core	2+ cores
RAM	512 MB	1+ GB
Disk	1 GB free	5+ GB
Network	Access to the hypervisor network	Same VLAN as the hypervisors

Software (installed automatically)

- **Node.js 20.x** — JavaScript runtime
- **PM2** — Node.js process manager
- **Nginx** — reverse proxy and web server

Proxmox Hypervisor

- **Proxmox VE 7.x or 8.x**
- **API Token** created with appropriate permissions
- **Port 8006** (HTTPS API) accessible from the VIBEVps server
- **QEMU Guest Agent** installed on VMs (recommended, required for SSH and IP detection)

Supported Browsers

- Chrome/Chromium 90+
- Firefox 90+
- Safari 15+
- Edge 90+

---

3. Installation

3.1 Server Preparation

Start from a clean Ubuntu 24.04 LTS installation. Ensure the server has network access to the Proxmox hypervisors (port 8006).

```
# Update the system
sudo apt update && sudo apt upgrade -y

# Verify connectivity to Proxmox
curl -k https://PROXMOX_IP:8006/api2/json/version
```

3.2 Automatic Installation

The recommended method is to use the automatic installation script included in the installer package.

```
# 1. Download and extract the installer package
unzip vibevps-installer-v1.0.0.zip
cd vibevps-installer-v1.0.0

# 2. Run the script as root
sudo bash install.sh
```

The script will automatically perform the following steps:

1. **OS verification** — confirms Ubuntu 24.04 LTS
2. **Information gathering** — asks for the Nginx port (default: 80), detects the server IP, generates JWT and encryption keys
3. **System update** — `apt update && apt upgrade`
4. **Node.js 20 installation** — via NodeSource repository
5. **PM2 installation** — process manager with autostart on boot
6. **Nginx installation** — reverse proxy + WebSocket configuration
7. **VIBEVps installation** — copies files, installs dependencies, builds, starts

Upon completion, a summary will be displayed with:

- Access URL (`http://SERVER_IP:PORT`)
- Default credentials (`admin` / `admin123!`)
- Useful PM2 commands

The installation log is saved to `/var/log/vibevps-install.log`.

3.3 Manual Installation

If you prefer a manual installation:

```
# 1. Install Node.js 20
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/nodesource.gpg
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/
  sudo tee /etc/apt/sources.list.d/nodesource.list
sudo apt update && sudo apt install -y nodejs

# 2. Install PM2
sudo npm install -g pm2
sudo pm2 startup systemd -u root --hp /root

# 3. Install Nginx
sudo apt install -y nginx

# 4. Create the application directory
sudo mkdir -p /var/www/vibevps
sudo mkdir -p /var/www/vibevps/data

# 5. Copy the application files
sudo cp -r backend frontend scripts package.json package-lock.json /var/www/vibevp

# 6. Create the .env file (see section 3.4)
sudo nano /var/www/vibevps/.env

# 7. Install dependencies and build
cd /var/www/vibevps
sudo npm install
sudo npm run build

# 8. Set permissions
sudo chown -R www-data:www-data /var/www/vibevps

# 9. Configure Nginx (see section 3.5)
# 10. Start with PM2 (see section 3.6)
```

3.4 Environment Configuration (.env)

Create the file `/var/www/vibevps/.env` with the following content:

```
# Server
PORT=3001
HOST=0.0.0.0

# JWT Secret (generate with: openssl rand -base64 32)
JWT_SECRET=YOUR_JWT_SECRET_KEY

# Database path (SQLite)
```

```
DB_PATH=/var/www/vibevps/data/vibevps.db

# CORS Origin (frontend access URL)
CORS_ORIGIN=http://SERVER_IP:NGINX_PORT

# Encryption key (generate with: openssl rand -hex 32)
ENCRYPTION_KEY=YOUR_ENCRYPTION_KEY
```

Variable	Description	Default
`PORT`	Fastify backend port	`3001`
`HOST`	Bind address	`0.0.0.0`
`JWT_SECRET`	Secret key for signing JWT tokens	Randomly generated
`DB_PATH`	SQLite database path	`data/vibevps.db`
`CORS_ORIGIN`	Frontend URL (for CORS)	`http://localhost:5173`
`ENCRYPTION_KEY`	AES-256-GCM key for credential encryption (prepared but not yet implemented)	Randomly generated

> **Important**: Always generate unique keys for `JWT_SECRET` and `ENCRYPTION_KEY` in production. Never use default values.

3.5 Nginx Configuration

Create the file `/etc/nginx/sites-available/vibevps`:

```
server {
    listen 80;
    server_name _;

    client_max_body_size 50M;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
        proxy_cache_bypass $http_upgrade;

        # WebSocket support
        proxy_connect_timeout 300s;
        proxy_send_timeout 300s;
        proxy_read_timeout 300s;
    }

    # WebSocket endpoint
    location /ws {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 86400;
    }
}
```

Enable the site and restart Nginx:

```
sudo ln -sf /etc/nginx/sites-available/vibevps /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl enable nginx
```

> **Note**: To configure HTTPS with an SSL certificate, add a `server` block on port 443 with `ssl_certificate` and `ssl_certificate_key` directives, and a 301 redirect from the port 80 block.

3.6 Starting with PM2

Create the PM2 ecosystem file at `/var/www/vibevps/ecosystem.config.cjs`:

```
module.exports = {
  apps: [{
    name: 'vibevps',
    script: 'backend/dist/server.js',
    cwd: '/var/www/vibevps',
    env_file: '.env',
    node_args: '--env-file=.env'
```

```
    }]
  };
```

Start and save the configuration:

```
cd /var/www/vibevps
sudo pm2 start ecosystem.config.cjs
sudo pm2 save
```

The application will be automatically restarted on server boot thanks to `pm2 startup`.

---

4. First Login

1. Open your browser and navigate to the server URL: `http://SERVER_IP:PORT`
2. Log in with the default credentials:
   - **Username**: `admin`
   - **Password**: `admin123!`
3. **Change the password immediately** from the Settings page

> **Warning**: The default credentials are the same for every installation. Changing them on first login is essential for security.

The JWT session lasts **8 hours**. After expiration, you will need to log in again.

---

5. Proxmox Hypervisor Configuration

5.1 Creating an API Token on Proxmox

Before adding a hypervisor in VIBEVps, you need to create an API Token in the Proxmox web interface:

1. Access the Proxmox web interface (`https://PROXMOX_IP:8006`)
2. Navigate to **Datacenter → Permissions → API Tokens**
3. Click **Add**
4. Configure:
   - **User**: select a user (e.g., `root@pam` or a dedicated user)

- **Token ID**: choose a name (e.g., `vibevps`)
- **Privilege Separation**: **uncheck** (the token inherits the user's permissions)

5. Click **Add** and **copy the secret** shown — it will not be visible again

The API Token format will be:

- **Token ID**: `user@realm!tokenid` (e.g., `root@pam!vibevps`)
- **Token Secret**: a UUID string (e.g., `aaaaaa-bbb-cccc-dddd-eeeeeeee`)

> **Required permissions**: the token needs at least permissions to read and manage VMs, storage, and nodes. If using `root@pam`, all permissions are already available.

5.2 Adding a Hypervisor in VIBEVps

1. Navigate to the **Hypervisors** page from the sidebar
2. Click **Add Hypervisor**
3. Fill in the form:

Field	Description	Example
Name	Identifying name	`Proxmox-Prod-01`
Type	Hypervisor type	`Proxmox`
Host	Proxmox server IP or hostname	`192.168.1.100`
Port	API port (default 8006)	`8006`
API Token ID	Token ID in `user@realm!tokenid` format	`root@pam!vibevps`
API Token Secret	The secret copied earlier	`aaaaaa-bbb-cccc-...`
Verify SSL	Verify SSL certificate (disable for self-signed certificates)	`No`

4. Click **Save**

> **Note**: The **Node** field is automatically detected during the connection test. It does not need to be entered manually.

5.3 Connection Test

After adding the hypervisor:

1. Click the **Test** button next to the hypervisor in the list
2. If the connection succeeds, the following will be displayed:
    - Proxmox node name
    - Proxmox VE version
    - Server hostname
3. The node name will be saved automatically

If the test fails, check:

- That the IP and port are correct
- That the firewall allows connections on port 8006
- That the API token is valid and not expired
- That the VIBEVps server has network connectivity to Proxmox

---

6. Dashboard

The Dashboard is the main page, visible immediately after login. It shows a complete overview of the status of all hypervisors and VMs.

6.1 Node Overview

For each connected hypervisor, a card is displayed with:

- **CPU**: real-time usage percentage with progress bar
- **RAM**: current usage / total with progress bar
- **Disk**: used space / total
- **Uptime**: node uptime

Data is automatically refreshed every **5 seconds** via WebSocket.

6.2 VM Statistics

A summary shows:

- Total number of VMs
- Running VMs
- Stopped VMs
- Paused/suspended VMs

6.3 Storage and Network Traffic

- **Storage**: list of storage configured on Proxmox with usage bars. You can hide/show specific storage from the Settings page.
- **Network traffic**: inbound and outbound speed calculated in real time from WebSocket data, with total traffic summary.

6.4 Hypervisor Filter

When **multiple hypervisors** are configured, a selector automatically appears at the top to filter the view to a single hypervisor or show all.

---

7. Virtual Machine Management

7.1 VM List

The **Virtual Machines** page shows all VMs from all hypervisors in a single view:

- **Search**: filter by VM name or ID
- **Status filter**: show only running, stopped VMs, etc.
- **Hypervisor filter**: when there are multiple hypervisors
- **Table view** (desktop): columns for name, ID, status, CPU, RAM, disk, hypervisor
- **Card view** (mobile): compact cards for each VM

Each row/card shows a colored badge for the status:

- **Running** — VM is running (green)
- **Stopped** — VM is stopped (red)
- **Paused** — VM is paused (yellow)
- **Suspended** — VM is suspended (orange)
- **Unknown** — Status cannot be determined (gray)

7.2 VM Details

Clicking on a VM opens the detail page showing:

- **General information**: name, VMID, status, parent hypervisor
- **Resources**: CPU core count, allocated RAM, disk size
- **Runtime metrics** (if running): uptime, CPU usage, network traffic
- **Action buttons**: start, stop, restart, suspend, delete
- **SSH button**: opens the SSH terminal in the browser (see section 10)

7.3 VM Actions

Available actions depend on the current VM status:

Action	From Status	Result
Start	Stopped, Suspended	Starts the VM
Stop	Running, Paused	Stops the VM (ACPI shutdown)
Restart	Running	Restarts the VM
Suspend	Running	Suspends the VM to RAM

Actions can be performed from both the VM list (inline buttons) and the detail page.

After each action, the page automatically refreshes after 2 seconds to show the new status.

7.4 VM Deletion

Deleting a VM requires explicit confirmation via a modal dialog:

1. Click the **Delete** button on the detail page
2. Confirm in the dialog window
3. If the VM is running, it will be automatically stopped first, then deleted
4. The operation is **irreversible**

---

8. Creating Virtual Machines

8.1 Prerequisites: Proxmox Templates

To create VMs through VIBEVps, you need at least one **VM template** configured on Proxmox:

1. Install a VM on Proxmox with the desired operating system
2. Install **cloud-init** and **qemu-guest-agent**:

```
# On Ubuntu/Debian
sudo apt install cloud-init qemu-guest-agent

# On CentOS/RHEL
sudo yum install cloud-init qemu-guest-agent
```

3. Configure the cloud-init disk in Proxmox (Hardware → Add → CloudInit Drive)
4. Convert the VM to a template: right-click → **Convert to Template**

8.2 Creation Wizard (5 Steps)

VM creation is done through a guided 5-step wizard, accessible from the **Create VM** page in the sidebar.

Step 1: Template Selection

- Select the destination hypervisor
- Choose a template from the list of saved templates
- Or discover templates directly from the hypervisor ("Discover" button)

Step 2: VM Identity

- **VM Name**: name of the VM (visible in Proxmox and on the dashboard)
- **Hostname**: hostname that will be configured via cloud-init inside the VM

Step 3: Network

Two modes available:

DHCP (default):

- The IP will be assigned automatically by the network's DHCP server
- VIBEVps automatically regenerates the `machine-id` to obtain a unique DHCP lease

Static IP:

- **IP Address**: IP address (e.g., `192.168.1.50`)
- **Netmask**: in CIDR format (e.g., `24` for `/24` i.e. `255.255.255.0`)
- **Gateway**: network gateway (e.g., `192.168.1.1`)
- **DNS Servers**: DNS servers separated by comma (default: `8.8.8.8, 1.1.1.1`)

Step 4: Resources

- **CPU Cores**: number of virtual cores (default from template)
- **RAM (MB)**: RAM in megabytes (default from template)
- **Disk (GB)**: disk size in gigabytes (default from template; the disk is expanded if a value larger than the template is requested)

Step 5: Summary

Review all configurations before confirming. Click **Create** to start the process.

8.3 Cloud-Init Configuration

VIBEVps uses **cloud-init** to automatically configure newly created VMs:

- **Hostname**: set according to what was specified in Step 2
- **Network**: static IP or DHCP configuration as specified in Step 3
- **DNS**: configured DNS servers
- **Guest Agent**: a vendor-data snippet is uploaded for auto-installation of `qemu-guest-agent` if not already present
- **Machine-ID**: automatically regenerated to avoid DHCP conflicts when cloning VMs

8.4 What Happens After Creation

The creation process executes sequentially:

1. **Clone** — Full clone (not linked) of the template
2. **Cloud-Init Drive** — Adding the cloud-init drive if missing
3. **Disk Resize** — Disk expansion if requested
4. **Unique MAC** — Generation of a unique MAC address (locally-administered)
5. **Configuration** — Applying hostname, IP, DNS, resources, guest agent
6. **Vendor-Data** — Uploading snippet for qemu-guest-agent auto-installation
7. **Start** — Starting the VM
8. **Post-Boot** — Waiting for guest agent (up to 120 seconds), then:
    - `machine-id` regeneration (for unique DHCP leases)
    - Filesystem expansion to use all available disk space

Upon completion, the VM will appear in the list with **Running** status.

---

9. Template Management

The **Templates** page allows you to save and organize VM templates for quick reuse.

9.1 Automatic Template Discovery

1. Select a hypervisor from the filter
2. Click **Discover Templates**
3. VIBEVps queries Proxmox and shows all VMs marked as templates
4. Select the templates to import

9.2 Saving Templates with Default Values

When saving a template, you can configure default values that will be pre-filled in the VM creation wizard:

Field	Description
Name	Descriptive name (e.g., "Ubuntu 24.04 Base")
Description	Free-form template description
Default Cores	Default number of CPU cores
Default Memory (MB)	Default RAM in MB
Default Disk (GB)	Default disk size in GB
OS Type	Operating system type (informational)

Saved templates will be available in Step 1 of the VM creation wizard.

---

10. Web SSH Console

VIBEVps includes a browser-based SSH terminal, built on **xterm.js** and a WebSocket proxy.

10.1 Requirements

To use the SSH console, the VM must:

- Be in **Running** state
- Have **qemu-guest-agent** active (for IP detection)
- Have an **SSH server** listening on port 22
- Have valid SSH credentials (username/password)

10.2 Connecting

1. From the detail page of a running VM, click the **SSH** button
2. A modal with the terminal opens
3. Enter the VM's SSH **username** and **password**
4. The terminal connects and displays the remote shell

The terminal supports:

- **Automatic resizing** to the browser window

- **Copy/paste** with standard terminal shortcuts
- All commands and interactive programs (vim, htop, etc.)

10.3 IP Resolution

VIBEVps determines the VM's IP using a fallback chain:

1. **QEMU Guest Agent** — queries the agent for network interface addresses (excludes loopback and link-local)
2. **VM Detail** — extracts the IP from the `ipconfig` field in the Proxmox configuration
3. **Cloud-Init Config** — reads the IP from the saved cloud-init configuration

---

11. Activity Logs

The **Logs** page shows the history of all operations performed on VMs:

Field	Description
Action	Type of operation (create, start, stop, restart, delete, suspend)
VM Name	Name of the affected VM
Hypervisor	Hypervisor where the action was performed
Status	Outcome (success / error)
Details	Additional details (in JSON format)
Timestamp	Date and time of the operation

Actions are displayed with colored badges:

- **Create** → green
- **Start** → blue
- **Stop** → red
- **Restart** → orange
- **Delete** → dark red
- **Suspend** → yellow

The **100 most recent logs** are shown.

---

12. Settings

12.1 Password Change

From the **Settings** page you can change the admin user's password:

1. Enter the **current password**
2. Enter the **new password**
3. Confirm and click **Change Password**

> **Important**: change the default password on first login.

12.2 Storage Visibility

The storage section in settings allows you to control which storage units are shown on the Dashboard:

- Each storage detected from the hypervisors has an on/off toggle
- Disabled storage will not be shown on the Dashboard
- This configuration is saved in the database and persists across sessions

---

13. Update System

VIBEVps includes a built-in update system that **does not require a license**.

13.1 Checking for Updates

1. Navigate to the **Updates** page from the sidebar
2. Click **Check for Updates**
3. If available, the following will be shown:
    - **Current version** installed
    - **New version** available
    - **Release date**
    - **Changelog** with the list of changes

13.2 Installing Updates

1. After checking for availability, click **Install Update**
2. The system will download the update package

3. The `update.sh` script will be executed in the background
4. The application will restart automatically
5. Reload the page to see the new version

> **Note**: during the update, the application may be temporarily unavailable for a few seconds during the restart.

13.3 Sequential Updates

If multiple intermediate versions are available (e.g., from 1.0.0 to 1.3.0), the system automatically applies updates sequentially, version by version. This ensures that any database migrations or structural changes are applied in the correct order.

13.4 Automatic Backup

Before each update, the script automatically creates a full backup in `/var/www/vibevps-backups/`:

- Backend and frontend code
- `.env` file
- SQLite database
- `package.json`

The **last 5 backups** are retained; older ones are automatically removed.

To restore a backup in case of issues:

```
# Stop the application
sudo pm2 stop vibevps

# Restore the backup
sudo cp -r /var/www/vibevps-backups/backup-YYYYMMDD_HHMMSS/* /var/www/vibevps/

# Rebuild and restart
cd /var/www/vibevps
sudo npm install && sudo npm run build
sudo pm2 restart vibevps
```

---

14. Technical Architecture

14.1 Technology Stack

Component	Technology
Backend	Fastify 5 + TypeScript (ESM)
Frontend	React 19 + Vite + TailwindCSS v4
Database	SQLite via better-sqlite3 (WAL mode)
Auth	JWT (jsonwebtoken) + bcrypt
Process Manager	PM2
Reverse Proxy	Nginx
Real-time	WebSocket (ws library)
SSH Terminal	xterm.js (frontend) + ssh2 (backend)
HTTP Client	undici (for Proxmox API)
Validation	Zod
State Management	Zustand
Routing	react-router-dom v7
UI Components	Radix UI + custom components

14.2 Project Structure

```
vibevps/
├── backend/
│   └── src/
│       ├── server.ts              # Fastify entry point
│       ├── config.ts              # Env vars configuration
│       ├── auth/
│       │   ├── auth.routes.ts     # Login, me, change-password
│       │   └── auth.guard.ts      # JWT middleware
│       ├── db/
│       │   ├── database.ts        # DB init, WAL, seed admin
│       │   └── schema.sql         # SQLite schema
│       ├── hypervisors/
│       │   ├── hypervisor.routes.ts
│       │   ├── hypervisor.service.ts
│       │   └── adapters/
│       │       ├── adapter.interface.ts  # Common interface
│       │       └── proxmox.adapter.ts    # Proxmox implementation
```

```
│       ├── vms/
│       │   └── vm.routes.ts       # CRUD + VM creation
│       ├── templates/
│       │   └── template.routes.ts
│       ├── settings/
│       │   └── settings.routes.ts
│       ├── updates/
│       │   ├── updates.routes.ts
│       │   ├── license.service.ts
│       │   └── update-sequencer.ts
│       └── ws/
│           ├── ws.hub.ts          # Real-time WebSocket hub
│           └── ssh.handler.ts     # SSH WebSocket proxy
├── frontend/
│   └── src/
│       ├── App.tsx                # Main routing
│       ├── api/
│       │   └── client.ts          # API client singleton
│       ├── pages/
│       │   ├── Login.tsx
│       │   ├── Dashboard.tsx
│       │   ├── VmList.tsx
│       │   ├── VmDetail.tsx
│       │   ├── CreateVm.tsx
│       │   ├── Hypervisors.tsx
│       │   ├── Templates.tsx
│       │   ├── Logs.tsx
│       │   ├── Settings.tsx
│       │   └── Updates.tsx
│       ├── components/
│       │   ├── layout/
│       │   │   ├── Layout.tsx     # Wrapper with responsive sidebar
│       │   │   └── Sidebar.tsx    # Navigation menu
│       │   └── ui/                # Reusable components
│       ├── store/
│       │   ├── auth.ts            # Authentication store (Zustand)
│       │   └── vms.ts             # VM store (Zustand)
│       └── hooks/
│           └── useWebSocket.ts    # Real-time WebSocket hook
├── installer/
│   ├── build-update.sh            # Update generation script
│   └── vibevps-installer-v1.0.0/
│       ├── install.sh             # Installation script
│       ├── config/
│       │   ├── .env.template
│       │   └── nginx-vibevps.conf
│       └── scripts/
│           └── update.sh          # Update script
├── data/
│   └── vibevps.db                 # SQLite database (generated)
```

```
├── docs/                          # Documentation
└── package.json                   # Root workspace
```

14.3 SQLite Database

The database uses SQLite in **WAL (Write-Ahead Logging)** mode for improved concurrency. The main tables are:

Table	Description
`users`	Users with hashed passwords (bcrypt)
`hypervisors`	Hypervisor configurations (host, API credentials)
`vm_logs`	Logs of all VM operations
`vm_templates`	Saved templates with default values
`settings`	Generic key-value settings
`license_info`	License information (optional)

The database is automatically created on first startup with the predefined schema and a default admin user.

Default path: `/var/www/vibevps/data/vibevps.db`

14.4 REST API

All APIs are prefixed with `/api` and protected by JWT authentication (except `/api/auth/login`).

Authentication

Method	Endpoint	Description
POST	`/api/auth/login`	Login with username/password → JWT
GET	`/api/auth/me`	Current user info
POST	`/api/auth/change-password`	Change password

Hypervisor

Method	Endpoint	Description

GET	`/api/hypervisors`	List hypervisors
POST	`/api/hypervisors`	Add hypervisor
PUT	`/api/hypervisors/:id`	Edit hypervisor
DELETE	`/api/hypervisors/:id`	Delete hypervisor
GET	`/api/hypervisors/:id/test`	Connection test
GET	`/api/hypervisors/:id/status`	Node status (CPU, RAM, disk)
GET	`/api/hypervisors/:id/storages`	Storage list with usage

Virtual Machines

Method	Endpoint	Description
GET	`/api/vms`	List all VMs
GET	`/api/vms/:hypervisorId/:vmId`	Single VM details
POST	`/api/vms/:hypervisorId/:vmId/:action`	VM action (start/stop/restart/suspend)
DELETE	`/api/vms/:hypervisorId/:vmId`	Delete VM
POST	`/api/vms/create`	Create VM from template
GET	`/api/vms/logs`	VM activity logs

Templates

Method	Endpoint	Description
GET	`/api/templates`	List saved templates
GET	`/api/templates/discover/:hypervisorId`	Discover templates on hypervisor
POST	`/api/templates`	Save template
DELETE	`/api/templates/:id`	Delete template

Settings

Method	Endpoint	Description
GET	`/api/settings`	All settings
GET	`/api/settings/:key`	Single setting
PUT	`/api/settings/:key`	Create/update setting

Updates and License

Method	Endpoint	Description
GET	`/api/updates`	Check for available updates
POST	`/api/updates/install`	Install update
GET	`/api/license`	License and server info
POST	`/api/license/activate`	Activate license
POST	`/api/license/verify`	Verify license

14.5 Real-time WebSocket

VIBEVps uses two WebSocket endpoints:

`/ws` — Real-time Hub

Connects automatically when the Dashboard is opened. Every **5 seconds** it sends an update with:

```
{
  "type": "update",
  "data": {
    "nodeStatus": { "cpu": 0.15, "memory": {...}, "disk": {...}, "uptime": 86400 }
    "vms": [ { "vmid": 100, "name": "web-01", "status": "running", ... } ],
    "storages": [ { "storage": "local-lvm", "used": 50, "total": 100, ... } ]
  }
}
```

The Dashboard, VM list, and Zustand stores receive and display this data in real time without needing to refresh.

`/ws/ssh` — SSH Proxy

Establishes an SSH proxy connection between the browser and the VM, allowing the use of a full terminal in the browser. Terminal data flows bidirectionally through the backend.

---

15. Maintenance and Troubleshooting

15.1 Useful PM2 Commands

```
# Application status
sudo pm2 status

# Real-time logs
sudo pm2 logs vibevps

# Restart application
sudo pm2 restart vibevps

# Stop application
sudo pm2 stop vibevps

# Resource monitoring (CPU/RAM)
sudo pm2 monit

# Reload after changes (zero-downtime)
sudo pm2 reload vibevps
```

15.2 System Logs

Log	Path
Application log	`~/.pm2/logs/vibevps-out.log` and `vibevps-error.log`
Installation log	`/var/log/vibevps-install.log`
Update log	`/var/log/vibevps-update.log`
Nginx log	`/var/log/nginx/access.log` and `error.log`

15.3 Database Backup

The SQLite database is a single file that can be copied directly:

```
# Manual backup
sudo cp /var/www/vibevps/data/vibevps.db /backup/vibevps-$(date +%Y%m%d).db

# Backup with cron script (daily at 2:00 AM)
echo "0 2 * * * root cp /var/www/vibevps/data/vibevps.db /backup/vibevps-\$(date +
  sudo tee /etc/cron.d/vibevps-backup
```

> **Note**: SQLite in WAL mode may have additional files (`-wal` and `-shm`). For a consistent backup, it is recommended to briefly stop the application or use SQLite's `.backup` command.

15.4 Common Issues and Solutions

Application won't start

```
# Check PM2 status
sudo pm2 status

# Check error logs
sudo pm2 logs vibevps --err --lines 50

# Verify port 3001 is free
sudo ss -tlnp | grep 3001

# Restart
sudo pm2 restart vibevps
```

Page unreachable (502 Bad Gateway)

```
# Verify Nginx is running
sudo systemctl status nginx

# Check the configuration
sudo nginx -t

# Verify the backend is listening
sudo pm2 status
curl http://127.0.0.1:3001/api/auth/me
```

```
# Restart everything
sudo pm2 restart vibevps && sudo systemctl restart nginx
```

Hypervisor unreachable

- Check network connectivity: `curl -k https://PROXMOX_IP:8006/api2/json/version`
- Check the firewall: `sudo ufw status` (on the VIBEVps server) and Proxmox firewall rules
- Verify the API token on the Hypervisors page → Edit → check Token ID and Secret
- Verify the token has not expired in the Proxmox interface

VM created but IP not detected

- Verify that `qemu-guest-agent` is installed and active in the VM: `systemctl status qemu-guest-agent`
- Verify that the `virtio-serial` device is present in the VM's hardware configuration in Proxmox
- For DHCP VMs: wait a few seconds after boot for the agent to become active
- Try starting the agent manually: `sudo systemctl start qemu-guest-agent`

SSH console won't connect

- Verify the VM is in **Running** state
- Verify the SSH service is active in the VM: `systemctl status sshd`
- Verify SSH credentials (correct username and password)
- Verify the VM's IP has been detected correctly (see previous point)
- Check for any firewall rules in the VM blocking port 22

Corrupted database

```
# Stop the application
sudo pm2 stop vibevps

# Try to repair
cd /var/www/vibevps/data
sqlite3 vibevps.db "PRAGMA integrity_check;"

# If corrupted, restore from backup
sudo cp /var/www/vibevps-backups/backup-LATEST/data/vibevps.db /var/www/vibevps/da

# Restart
sudo pm2 restart vibevps
```

---

16. Security

Recommendations

1. **Change the default password** immediately after installation
2. **Generate unique keys** for `JWT_SECRET` and `ENCRYPTION_KEY` in the `.env` file
3. **Configure HTTPS** with a valid SSL certificate (Let's Encrypt or commercial certificate) to encrypt traffic
4. **Restrict network access** to the VIBEVps panel to authorized IPs only via firewall
5. **Update regularly** the operating system and VIBEVps

Implementation Details

Aspect	Implementation
Authentication	JWT with 8-hour expiry
Passwords	bcrypt hash (cost factor 10)
Auth Guard	Middleware on all protected routes
Hypervisor Credentials	Stored in SQLite (AES-256-GCM encryption prepared but not yet implemented)
CORS	Configurable via environment variable
Proxmox SSL	Optional verification (recommended to disable only for self-signed certificates)
Token localStorage	Saved as `vvps_token`, automatically removed on logout or expiry

---

17. Local Development

To contribute to development or run VIBEVps locally:

```
# 1. Clone the repository
git clone https://github.com/user/vibevps.git
cd vibevps

# 2. Install dependencies (npm workspaces)
npm install
```

```
# 3. Start in development mode (backend + frontend)
npm run dev
```

The application will be available at:

- **Frontend**: `http://localhost:5173` (Vite dev server with hot reload)
- **Backend**: `http://localhost:3001` (Fastify with tsx watch)

The Vite proxy automatically redirects `/api` and `/ws` calls to the backend.

Available Commands

```
npm run dev              # Backend (:3001) + Frontend (:5173) in parallel
npm run dev:backend      # Backend only with hot reload
npm run dev:frontend     # Frontend only with hot reload
npm run build            # Production build (frontend + backend)
cd frontend && npm run lint  # Lint frontend code
npm run start            # Start in production mode
```

Workspace Structure

The project uses npm workspaces with two packages:

- `backend/` — Fastify + TypeScript (ESM, imports with `.js` extension)
- `frontend/` — React + Vite + TailwindCSS v4

The `@/` path alias in the frontend maps to `frontend/src/`.

Manual generated for VIBEVps v1.3.1 — March 22, 2026